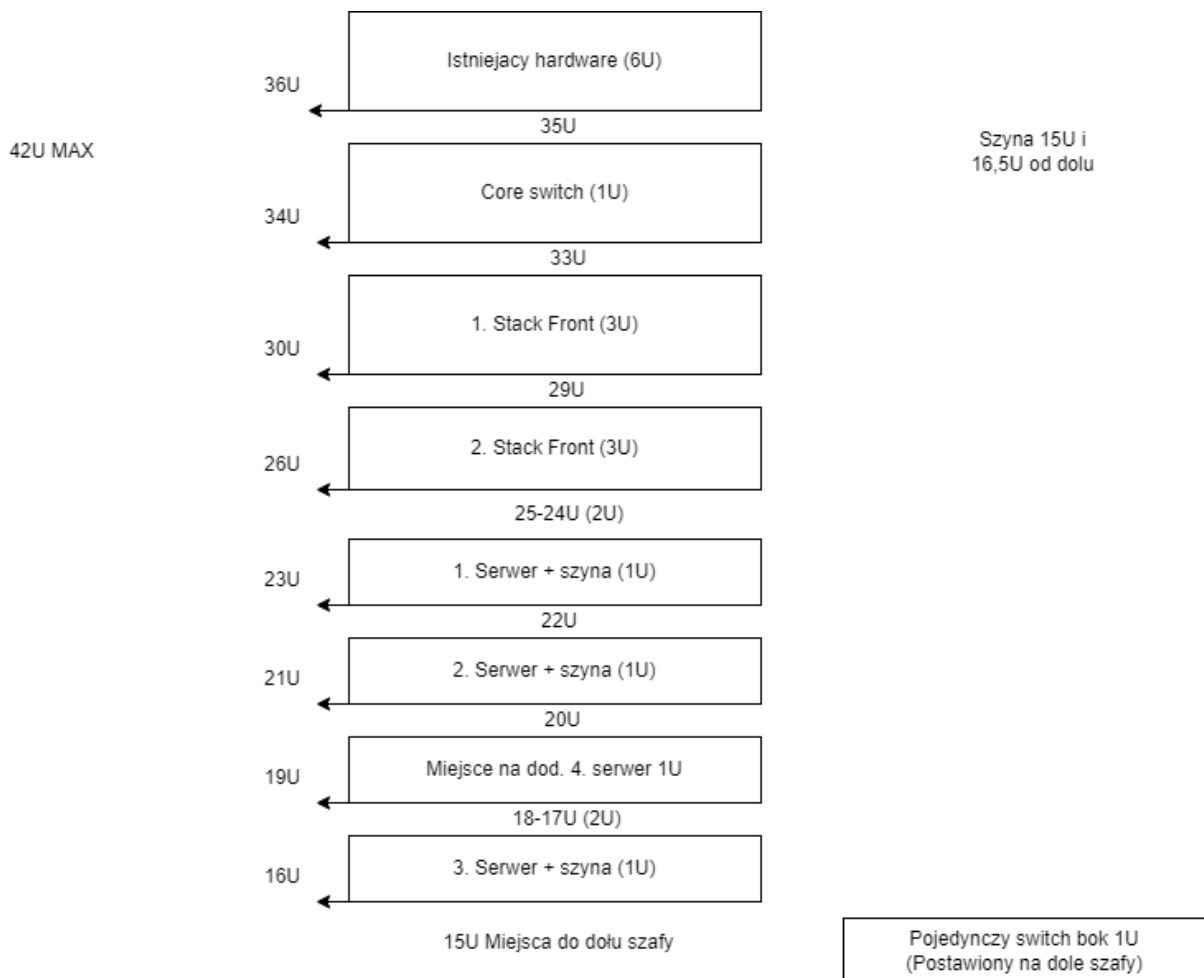


Sprawozdanie z ASSK – Grupa Szafa

1.Wstęp, opis problemu i projekt szafy

Sprawozdanie dotyczy konfiguracji lokalnej infrastruktury sieciowej i serwerowej przygotowywanej w ramach zajęć z administracji systemami Linux oraz sieci komputerowych Cisco. Scenariusz zakładał stworzenie środowiska przypominającego infrastrukturę małej lub średniej firmy, która potrzebuje własnej, lokalnej sieci, podziału na zespoły, usług katalogowych, usług plikowych, systemów bezpieczeństwa, hostingu, backupu oraz narzędzi produktywności. W praktyce oznaczało to konieczność połączenia kilku obszarów administracji IT: pracy z fizycznym sprzętem w szafie rack, konfiguracji przełączników Cisco, przygotowania serwerów Linux do działania jako routery programowe, uruchomienia usług DHCP, DNS, VPN i firewalla, a także integracji z usługami przygotowywanymi przez inne zespoły, przede wszystkim z Active Directory oraz mechanizmami uwierzytelniania sieciowego.

W projekcie wykorzystywany był sprzęt sieciowy i serwerowy dostępny w pracowni, m.in. dwa przełączniki Cisco SG500X-48P, dodatkowy switch Cisco Catalyst C2960, core switch Cisco Catalyst C4500-X-16SFP+, serwer Dell PowerEdge R310 oraz zasilacze rackowe. Sam serwer PowerEdge R310 zajmował 1U, ale ze względu na swoją głębokość wymagał montażu na szynach, co miało znaczenie przy projektowaniu fizycznego układu szafy.



Pierwszym etapem prac było uporządkowanie infrastruktury fizycznej. Szafa Rack początkowo była pusta a jej odpowiednie wypełnienie sprzętem stanowiło także ważny element całego projektu. Trzeba było rozplanować położenie przełączników, serwerów, istniejącego sprzętu oraz przestrzeni przeznaczonej na dalszą rozbudowę. Przyjęty układ zakładał maksymalną wysokość szafy 42U, pozostawienie miejsca na istniejący hardware, core switch, dwa przełączniki pracujące jako stack z frontu szafy, dwa serwery na szynach rail oraz dodatkową przestrzeń na kolejny serwer oraz inny sprzęt w przyszłości. To było ważne, ponieważ infrastruktura miała być nie tylko uruchomiona, ale również posiadać możliwość do dalszego rozwoju i serwisowania. Aby to osiągnąć rozmontowano wcześniejszą zawartość szafy, ponownie zamontowano serwery i przełączniki zgodnie z projektem, a następnie połączono kablami (mowa głównie o RJ45). Był to etap czasochłonny, ponieważ część sprzętu była ciężka, część wymagała dodatkowych elementów montażowych, a dokumentacja urządzeń, szczególnie tych starszych lub wycofanych ze wsparcia, nie była łatwa do odnalezienia.

2.Segmentacja i zabezpieczanie sieci lokalnej

ISP: 10.3.0.153/24

DNS: 192.168.1.11, 192.168.1.3, 8.8.8.8

AD: 192.168.1.11

DHCP: 192.168.1.1, 192.168.1.2

VRRP: adres wirtualny 192.168.1.3 dla mastera (192.168.1.1) oraz secondary (192.168.1.2)

Bezpieczeństwo hosty: 192.168.1.21, 192.168.1.22, 192.168.1.23 (wazuh)

Segmentacja adresów IP dla każdej grupy

192.168.1.1 - 192.168.1.9 - Grupa związana z administracją szafy rack i siecią

192.168.1.11 - 192.168.1.19 - Grupa związana z AD

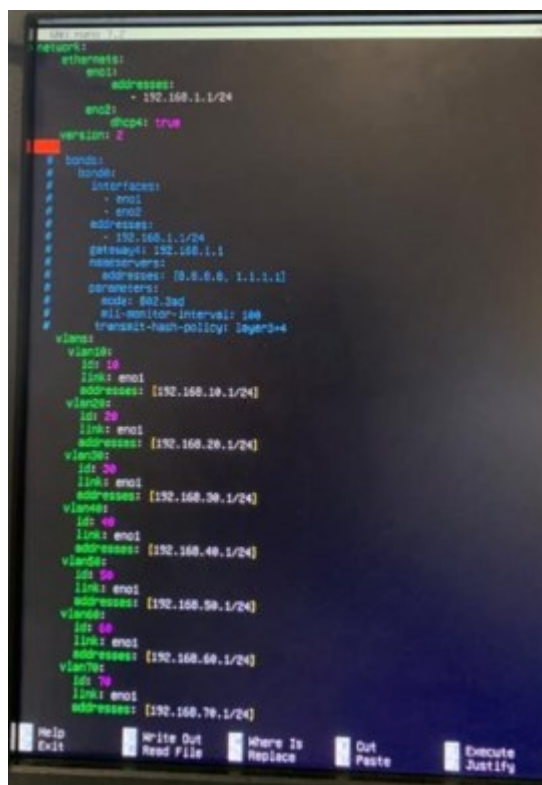
192.168.1.21 - 192.168.1.29 - Grupa związana z cyberbezpieczeństwem (raporty, skany, przyleganie do standardów + SIEM)

192.168.1.31 - 192.168.1.39 - Grupa związana z backupami + SMB

192.168.1.41 - 192.168.1.49 - Grupa związana z chmurą + mailami + hostingiem WWW

Grupa produktywność nie dostała swoich adresów IP dla maszyn VM (nie potrzebowali)

192.168.1.50 - 192.168.1.255 - Adresy IP dla DHCP stacji roboczych



```
network:
  ethernet:
    ens1:
      addresses:
        - 192.168.1.1/24
    ens2:
      dhcp: true
  version: 2
  bonds:
    bond0:
      interfaces:
        - ens1
        - ens2
      addresses:
        - 192.168.1.1/24
      gateway4: 192.168.1.1
      nameservers:
        addresses: [8.8.8.8, 1.1.1.1]
      parameters:
        mode: 802.3ad
        xmit-monitor-interval: 100
        transmit-hash-policy: layer3+4
  vlans:
    vlan10:
      id: 10
      link: ens1
      addresses: [192.168.10.1/24]
    vlan20:
      id: 20
      link: ens1
      addresses: [192.168.20.1/24]
    vlan30:
      id: 30
      link: ens1
      addresses: [192.168.30.1/24]
    vlan40:
      id: 40
      link: ens1
      addresses: [192.168.40.1/24]
    vlan50:
      id: 50
      link: ens1
      addresses: [192.168.50.1/24]
    vlan60:
      id: 60
      link: ens1
      addresses: [192.168.60.1/24]
    vlan70:
      id: 70
      link: ens1
      addresses: [192.168.70.1/24]
```

Głównym zadaniem mojej części projektu była konfiguracja segmentacji sieci i zabezpieczeń na przełącznikach Cisco oraz przygotowanie warstwy routingu i usług sieciowych na dwóch serwerach Linux. Infrastruktura została podzielona logicznie na oddzielne grupy odpowiadające zespołom i usługom. Każda grupa miała otrzymać własną przestrzeń adresową, własne porty dostępowe na switchu oraz własny VLAN. Celem było uniknięcie sytuacji, w której wszystkie urządzenia znajdują się w jednej wspólnej sieci, bez separacji ruchu i bez możliwości kontroli dostępu. Segmentacja przez VLAN-y pozwalała rozdzielić ruch administracyjny, usługi Active Directory, backup, hosting/chmurę, bezpieczeństwo oraz produktywność.

```
Port 1 - internet
Port 2 - do debugowania
Port 3 - 4 dla grupy szafa
Port 5 - 8 dla grupy AD
Port 9 - 12 dla grupy backup
Port 13 - 16 dla grupy chmura
Port 17 - 20 dla grupy bezpieczeństwo
Port 21 - 23 dla grupy produktywność
Port 24 jest nieprzypisany
```

Na pierwszym przełączniku przyjęto plan, w którym port 1 był trunk przeznaczony na internet, port 2 na debugowanie i konfigurację, porty 3–4 dla grupy szafa, porty 5–8 dla grupy AD, porty 9–12 dla backupu, porty 13–16 dla grupy chmura, porty 17–20 dla bezpieczeństwa, porty 21–23 dla produktywności, a port 24 pozostał nieprzypisany.

Konfiguracja przełączników Cisco SG500X wymagała zarówno utworzenia VLAN-ów, jak i przypisania odpowiednich portów do trybu access. W praktyce port dostępowy oznaczał, że urządzenie końcowe podłączone do danego portu nie musiało samodzielnie tagować ramek VLAN, ponieważ switch przypisywał je do odpowiedniego VLAN-u na podstawie konfiguracji portu. Dzięki temu zwykła stacja robocza mogła zostać przepięta z portu należącego do jednej grupy do portu należącego do innej grupy i po odnowieniu dzierżawy DHCP otrzymywała adres z innej puli. Oprócz access portów zaplanowano również trunk, czyli port przenoszący wiele VLAN-ów jednocześnie, co było potrzebne do połączenia przełączników oraz do przekazywania ruchu z wielu segmentów do serwera pełniącego rolę routera.

W konfiguracji używano m.in. poleceń tworzących VLAN-y 10, 20 i kolejne, ustawiających porty GE 1/0/3–4 jako access dla VLAN 10, porty GE 1/0/5–8 jako access dla VLAN 20 itd... oraz konfigurujących trunk z listą dozwolonych VLAN-ów. W dokumentacji komend pojawia się również konfiguracja DHCP snooping, portfast, BPDU guard i storm-control, czyli mechanizmów podnoszących bezpieczeństwo i stabilność portów dostępowych.

Ważnym elementem konfiguracji było również zabezpieczenie samych przełączników w tym podjąć działania aby ograniczyć ryzyko przypadkowej lub nieautoryzowanej zmiany konfiguracji. Wykonano podstawowe czynności takie jak ustawienie hasła do trybu uprzywilejowanego, zabezpieczenie dostępu przez konsolę, ustawienie timeoutu sesji oraz przygotowanie dostępu zdalnego przez SSH tam, gdzie było to wspierane przez dany model i wersję oprogramowania (SSH lub Telnet). Część poleceń znanych z innych urządzeń Cisco nie była jednak obsługiwana przez switch SG500X albo działała inaczej przez starą wersję iOS, co było jednym z istotnych problemów praktycznych w szukaniu faktycznie działających komend. Przykładowo w dokumentacji Cisco zapisano, że

ograniczenie dostępu do VTY czy szyfrowanie haseł nie były wspierane przez używany model w oczekiwany sposób. To wymagało ostrożnego podejścia, testowania poleceń metodą prób i błędów, co było czasochłonne, sprawdzania, które funkcje rzeczywiście są dostępne na konkretnym urządzeniu, zamiast zakładania, że każdy switch Cisco obsługuje je.

3. Software router + usługi na serwerze

Drugim kluczowym obszarem była konfiguracja dwóch serwerów rackowych z systemem Linux. Na obu serwerach zainstalowano Ubuntu Server LTS 2022, skonfigurowano sprzętowy RAID 10 oraz iDRAC oraz przygotowano serwery do pracy jako routery programowe. Oznaczało to, że serwer nie pełnił roli samego hosta aplikacyjnego, ale odpowiadał także za przekazywanie pakietów między sieciami (forwarding), NAT masquerade. Udało się uczynić z serwerów wirtualne routery przez forwarding i NAT masquerade, uruchomić DHCP, utworzyć VLAN-y, access porty, plan połączeń, skonfigurować redundancję przez keepalived oraz ustawić korespondencję między VLAN-ami na przełącznikach i pulami DHCP na serwerze. Ważnym elementem było również zwirtualizowanie jednej fizycznej karty sieciowej na wiele podinterfejsów VLAN, tak aby każdy VLAN miał osobny interfejs logiczny po stronie Linuxa oraz własny adres bramy.



```
# cat /etc/dhcp/dhcpd.conf

subnet 192.168.1.0 netmask 255.255.255.0 {
  range 192.168.1.100 192.168.1.200;
  option routers 192.168.1.1;
  option domain-name-servers 192.168.1.10, 1.1.1.1;
}

subnet 192.168.10.0 netmask 255.255.255.0 {
  range 192.168.10.3 192.168.10.14;
  option routers 192.168.20.1;
  option domain-name-servers 1.1.1.1;
}

subnet 192.168.20.0 netmask 255.255.255.0 {
  range 192.168.20.3 192.168.20.14;
  option routers 192.168.20.1;
  option domain-name-servers 1.1.1.1;
}

subnet 192.168.30.0 netmask 255.255.255.0 {
  range 192.168.30.3 192.168.30.14;
  option routers 192.168.30.1;
  option domain-name-servers 1.1.1.1;
}

subnet 192.168.40.0 netmask 255.255.255.0 {
  range 192.168.40.3 192.168.40.14;
  option routers 192.168.40.1;
  option domain-name-servers 1.1.1.1;
}

subnet 192.168.50.0 netmask 255.255.255.0 {
  range 192.168.50.3 192.168.50.14;
  option routers 192.168.50.1;
  option domain-name-servers 1.1.1.1;
}

subnet 192.168.60.0 netmask 255.255.255.0 {
  range 192.168.60.3 192.168.60.14;
  option routers 192.168.60.1;
  option domain-name-servers 1.1.1.1;
}

subnet 192.168.70.0 netmask 255.255.255.0 {
  range 192.168.70.3 192.168.70.14;
  option routers 192.168.70.1;
  option domain-name-servers 1.1.1.1;
}

# host svr1 {
#
#   Help
#   Exit
#   Write Out
#   Read File
#   Where Is
#   Replace
#   Cut
#   Paste
# }
```

Konfiguracja DHCP była jednym z najważniejszych elementów całego rozwiązania, ponieważ właśnie ona pozwalała powiązać logiczną segmentację VLAN z praktycznym działaniem stacji roboczych. Dla każdego VLAN-u trzeba było przygotować osobną pulę adresową, wskazać właściwą maskę, bramę domyślną oraz serwery DNS. Dzięki temu komputer podłączony do portu przypisanego do konkretnego VLAN-u otrzymywał adres z odpowiedniej podsieci i mógł korzystać z usług właściwych dla danej grupy. To było szczególnie ważne w scenariuszu testowym, w którym przełączenie kabla Ethernet między portami należącymi do różnych VLAN-ów miało skutkować przydzieleniem nowego adresu IP i możliwością dołączenia do środowiska Active Directory. Na poziomie Linuxa wymagało to

zgodności między konfiguracją switcha, konfiguracją podinterfejsów VLAN w Netplanie oraz konfiguracją serwera DHCP. Jeżeli którykolwiek z tych elementów był niespójny, komputer mógł nie otrzymać adresu albo dostać adres z nieprawidłowej puli.

Osobnym elementem było przygotowanie routingu oraz NAT. Ponieważ serwery działały jako software routery, należało włączyć przekazywanie pakietów w jądrze Linuxa, skonfigurować interfejs zewnętrzny prowadzący do sieci ISP oraz interfejsy wewnętrzne lub podinterfejsy VLAN prowadzące do poszczególnych segmentów. NAT masquerade pozwalał hostom z sieci wewnętrznych korzystać z dostępu do internetu przez adres zewnętrzny routera. Jednocześnie routing między VLAN-ami musiał być kontrolowany, ponieważ sama możliwość routingu nie oznacza, że wszystkie segmenty powinny mieć pełny dostęp do wszystkich usług. Z tego powodu zastosowano reguły firewalla, które miały ograniczać ruch do potrzebnych portów i adresów. Dla przykładu grupa bezpieczeństwa korzystająca z Wazuh wymagała określonych portów: 443 TCP dla dashboardu, 1514–1515 TCP do komunikacji z agentami, 514 TCP jako syslog collector oraz 9200 TCP dla indexera. Takie informacje były podstawą do tworzenia reguł dopuszczających ruch tylko tam, gdzie był on uzasadniony funkcjonalnie.

4. Wybor firewalla

1. iptables (i spółka) albo nftables

- Podstawowe komendy/technologie wbudowane w większość linuksów (w tym w Ubuntu Server)
- Składnia nie jest przyjemna ale daje możliwości ustawiania podstawowych reguł
 - Składnie można uprościć poprzez stosowanie programów nakładek jak **ufw** czy **firewalld** (w wyniku działania komend pojawiają się reguły akceptowane przez **iptables/nftables**)
- Pozwala na port forwarding, sterowanie NAT, filtrowanie pakietów (po adresach IP, portach i protokołach, kierunku i interfejsach), ustawianie limitów szybkości/połączeń oraz monitorowania i logowania ruchu
 - Można rozszerzyć o komendę **ipsets** która by aplikowała automatycznie blacklisty z plików
- Nie wspiera domyślnie tworzenia reguł na końcówkach (endpoint) HTTP i aplikacjach na warstwie 7
 - Można rozszerzyć o **ModSecurity**, który implementuje funkcjonalność WAF na serwerach HTTP/S
- Nie posiada dedykowanych rozwiązań do zabezpieczeń Anty-DDoS ani potrzeb chmurowych: podziału na strefy, lepszego wsparcia VLAN i VXLAN etc, kontroli ACL, HA, ...
- Nie mają wbudowanego narzędzia do centralnego przechowywania reguł ani ich synchronizacji ich pomiędzy urządzeniami

Wybór technologii firewalla również był częścią analizy. Rozważano różne rozwiązania, m.in. iptables/nftables, firewalld, OPNsense/pfSense, VyOS oraz IPFire.

Ostatecznie najbardziej naturalnym wyborem dla istniejącej konfiguracji były iptables lub nftables, ponieważ są to technologie wbudowane w większość dystrybucji Linuxa, w tym w Ubuntu Server. Ich zaletą jest możliwość filtrowania pakietów po adresach IP, portach, protokołach, kierunku ruchu i interfejsach, a także obsługa NAT, port forwarding, limitowanie połączeń oraz logowanie ruchu. Ich wadą jest mniej przyjazna składnia oraz brak gotowych funkcji znanych z rozbudowanych firewalli aplikacyjnych lub rozwiązań klasy UTM.

Rozważano też firewalld, ze względu na obsługę stref plus obsługę prawie tych samych możliwości co iptables, lecz finalnie obsługa stref nie okazała się aż tak potrzebna. Dodano obsługę rozszerzającą iptables o obsługę reguł związanych z wykrywaniem z jakiego kraju pochodzi dany pakiet (reguły + lookup z GEO IP) oraz odmawiania pakietom z krajów wrogich.

Analiza bardziej rozbudowanych rozwiązań, takich jak OPNsense, pfSense, VyOS czy IPFire, pokazała, że mogłyby one zapewnić większą wygodę zarządzania, obsługę zaawansowanych reguł, integrację z

IDS/IPS, captive portal, VPN, proxy, ACL, rozwiązania anty-DDoS lub mechanizmy wysokiej dostępności. Z drugiej strony w naszym przypadku były one ocenione jako przerost formy nad treścią, ponieważ serwery Ubuntu były już postawione, routing działał, DHCP było przygotowane, a głównym celem projektu było zrozumienie działania sieci i usług, a nie wdrożenie gotowej dystrybucji firewall/router. OPNsense i pfSense dawałyby wygodny panel webowy i wsparcie CARP, VyOS oferowałby bardzo szerokie możliwości enterprise, ale wymagałby dodatkowej nauki oraz migracji z już przygotowanego środowiska. IPFire również był ciekawą alternatywą, ale brak wygodnego CLI i konieczność zarządzania panelem webowym były uznane za ograniczenie w kontekście zajęć i pracy na istniejących serwerach Ubuntu. Prawdopodobnie gdyby zostało przeznaczone więcej czasu na zajęcia to podjęta byłaby decyzja o instalacji VM z VyOS lub IPFire jako lepszego firewalla.

5. Redundancja routerów przez Keepalived (VRRP)

Konfiguracja keepalived (VRRP)

```
vrrp_instance VI_1 {
    state MASTER
    interface eno2
    virtual_router_id 51
    priority 150
    advert_int 1
    authentication {
        auth_type PASS
        auth_pass securepassword
    }
    virtual_ipaddress {
        192.168.1.3/24
    }
}
```

Bardzo istotnym elementem projektu była redundancja routerów. Ponieważ w środowisku znajdowały się dwa serwery, ich rola nie powinna ograniczać się do niezależnych maszyn pracujących osobno. Celem było stworzenie układu, w którym jeden serwer może pełnić rolę głównego routera, a drugi rolę zapasową. Do tego użyto keepalived i protokołu VRRP. W praktyce oznaczało to przygotowanie wirtualnego adresu IP, pod którym klienci widzą bramę domyślną. Jeżeli serwer główny przestanie działać, adres wirtualny może zostać przejęty przez serwer zapasowy. W przykładowej konfiguracji keepalived użyto instancji VI_1, interfejsu eno2, virtual_router_id 51, priorytetu 150, interwału ogłoszeń 1 sekunda, prostego uwierzytelniania hasłem oraz wirtualnego adresu 192.168.1.3/24. Dzięki temu infrastruktura nie była zależna wyłącznie od jednego serwera, a klienci mogli korzystać z jednej, stabilnej bramy logicznej.

6. Integracja z AD, DNS, NPS (802.1X RADIUS)

Kolejnym ważnym obszarem była integracja z Active Directory i NPS, czyli usługami przygotowywanymi przez inną grupę. Active Directory pełniło rolę centralnej usługi katalogowej, a DNS z AD był istotny dla poprawnego działania domeny. Z tego powodu w konfiguracji DHCP należało wskazać odpowiednie serwery DNS, tak aby stacje robocze mogły odnaleźć kontroler domeny i poprawnie dołączyć do domeny. Dodatkowo pojawiła się integracja z NPS i 802.1X, co w praktyce oznaczało możliwość powiązania dostępu do sieci z mechanizmem uwierzytelniania użytkowników lub urządzeń. W planie prac uwzględniono konfigurację 802.1X dla grupy AD, a w końcowym efekcie udało się osiągnąć stan, w którym stacja robocza po podłączeniu do odpowiedniego portu otrzymywała adres IP z właściwej puli DHCP i mogła komunikować się z usługami domenowymi.

Osiągnięty wynik był praktycznie bardzo istotny, ponieważ eksperyment potwierdzał, że segmentacja sieci realnie działała i była odporna na chwilowe zmiany w łączności. Test polegał na przepięciu kabla Ethernet wybranej stacji roboczej między portami należącymi do różnych VLAN-ów. Po zmianie portu komputer powinien odnowić dzierżawę DHCP, otrzymać adres z innej puli, dostać właściwą bramę i DNS, a następnie zachować możliwość komunikacji z usługami potrzebnymi w danym segmencie, w tym z Active Directory. Osiągnięcie takiego efektu wymagało poprawnej konfiguracji kilku warstw jednocześnie: fizycznego połączenia RJ45, portu access na switchu, wybranego VLAN-u switcha, portu trunk do routera, podinterfejsu VLAN na Linuxie, puli adresów oraz dzierżaw DHCP, reguł routingu, reguł NAT, reguł firewalla oraz DNS. Właśnie dlatego projekt był wartościowy, ponieważ pokazywał, że w realnej infrastrukturze sieciowej problem nie musi leżeć wyłącznie w jednym miejscu. Błąd w adresacji, trunku, firewallu lub DNS może wyglądać podobnie z perspektywy użytkownika końcowego, który po prostu nie ma sieci albo nie może dołączyć do domeny, więc uczy to umiejętności diagnozowania problemów, wnioskowania i stopniowej eliminacji przyczyn.

W trakcie prac pojawiło się wiele problemów technicznych. Rozmontowanie szafy zajęło dużo czasu, core switch był trudny do fizycznego montażu bo był ciężki, notorycznie brakowało kabli, a część przełączników nie działała poprawnie i prawdopodobnie wymagała ponownego flashowania oprogramowania aby działała ponownie. Problemem było również to, że dokumentacja sprzętu EOL była trudniejsza do znalezienia, a wiele poradników internetowych i podpowiedzi AI sugerowało komendy, których używane switchy ze zbyt starym iOS nie obsługiwały. Po stronie Linuxa problemem był interfejs karty sieciowej, który nie pojawiał się jako aktywny, co utrudniało uruchomienie DHCP i testowanie ruchu. Zdarzały się również problemy z komputerami w poszczególnych VLAN-ach, które początkowo nie otrzymywały właściwych adresów z serwera DHCP albo zawieszały się podczas pobierania konfiguracji.

Problemy napotkane podczas zajęć

Największym wyzwaniem technicznym była konieczność zsynchronizowania konfiguracji wielu elementów naraz. VLAN utworzony na switchu musiał odpowiadać podinterfejsowi VLAN na routerze Linux. Port access musiał wskazywać właściwy VLAN. Trunk musiał przenosić listę dozwolonych VLAN-ów. DHCP musiało mieć osobną pulę dla każdego segmentu. Gateway w DHCP musiał wskazywać adres właściwego podinterfejsu lub adres wirtualny VRRP. DNS musiał prowadzić do AD, a firewall nie mógł blokować usług niezbędnych do działania domeny. Dodatkowo mechanizmy bezpieczeństwa switchy, takie jak DHCP snooping, portfast, BPDU guard i storm-control, musiały być skonfigurowane tak, aby zwiększać bezpieczeństwo, ale nie blokować prawidłowego ruchu.

Podsumowanie

Podsumowując, w ramach projektu udało się przygotować działającą bazę lokalnej infrastruktury IT opartej na przełącznikach Cisco i serwerach Linux. Zrealizowano fizyczne uporządkowanie szafy rack, konfigurację przełączników, utworzenie VLAN-ów, przypisanie portów do grup, konfigurację trunków, uruchomienie serwerów Ubuntu jako routerów programowych, przygotowanie DHCP, routingu, NAT, firewalla, DNS relay/fallback oraz redundancji z użyciem keepalived i VRRP. Dodatkowo rozwiązanie zostało zintegrowane z Active Directory i NPS/802.1X, co pozwoliło testować dostęp stacji roboczych do domeny w zależności od przypisanego segmentu sieci. Najważniejszym wynikiem było osiągnięcie sytuacji, w której przełączenie komputera do portu przypisanego do innego VLAN-u skutkowało przydzieleniem nowego adresu IP z odpowiedniej puli DHCP oraz zachowaniem możliwości komunikacji z usługami domenowymi. Projekt pokazał też realne trudności administracyjne: ograniczenia starszego sprzętu, problemy z dokumentacją, różnice między modelami Cisco, konieczność diagnozowania interfejsów sieciowych w Linuxie oraz potrzebę bardzo dokładnego pilnowania zgodności konfiguracji między warstwą fizyczną, przełącznikami, routingiem, DHCP, DNS i firewalliem.

Spis ważnych wykorzystanych komend w Cisco IOS CLI:

```
enable
configure terminal
enable password <hasło>
```

```
line console
password <hasło>
login
exit
```

```
ip routing
```

```
vlan database
vlan 10
name VLAN10
vlan 20
name VLAN20
vlan 30
name VLAN30
vlan 40
name VLAN40
vlan 50
name VLAN50
vlan 60
name VLAN60
vlan 70
name VLAN70
exit
```

```
configure terminal
```



```
interface range GE 1/1/3 - 4
switchport mode access
switchport access vlan 10
spanning-tree portfast
spanning-tree bpduguard enable
exit
```

```
interface range GE 1/1/5 - 8
switchport mode access
switchport access vlan 20
spanning-tree portfast
spanning-tree bpduguard enable
exit
```

```
interface range GE 1/1/9 - 12
switchport mode access
switchport access vlan 30
spanning-tree portfast
spanning-tree bpduguard enable
exit
```

```
interface range GE 1/1/13 - 16
switchport mode access
switchport access vlan 40
spanning-tree portfast
spanning-tree bpduguard enable
exit
```

```
interface range GE 1/1/17 - 20
switchport mode access
switchport access vlan 50
spanning-tree portfast
spanning-tree bpduguard enable
exit
```

```
interface range GE 1/1/21 - 23
switchport mode access
switchport access vlan 60
spanning-tree portfast
spanning-tree bpduguard enable
exit
```

```
interface range GE 1/1/3 - 23
storm-control broadcast level 70
storm-control include-multicast
exit
```

```
ip dhcp snooping
ip dhcp snooping vlan 10,20,30,40,50,60,70
```

```
interface GE 1/1/1
ip dhcp snooping trust
exit
```

```
interface GE 1/1/1
switchport mode trunk
switchport trunk allowed vlan add 10,20,30,40,50,60,70
ip dhcp snooping trust
exit
```

```
radius-server host 192.168.1.10 auth-port 1812 acct-port 1813 key
<wspolny_sekret_RADIUS>
```

```
dot1x system-auth-control
```

```
interface range GE 1/1/3 - 23
dot1x host-mode single-host
dot1x port-control auto
exit
```

```
interface range GE 1/1/3 - 23
port security
port security max 1
port security mode max-addresses
port security discard trap 60
exit
```

```
write memory
```

Fragmenty plików konfiguracyjnych:

/etc/sysctl.conf

```
net.ipv4.ip_forward=1
```

/etc/netplan/00-installer-config.yaml

```
network:
version: 2
renderer: networkd
```

```
ethernets:
eno1:
dhcp4: true
```

```
eno2:
dhcp4: false
```

```
vlan:
vlan10:
```

id: 10
link: eno2
addresses:
- 192.168.10.1/24

vlan20:
id: 20
link: eno2
addresses:
- 192.168.20.1/24

vlan30:
id: 30
link: eno2
addresses:
- 192.168.30.1/24

vlan40:
id: 40
link: eno2
addresses:
- 192.168.40.1/24

vlan50:
id: 50
link: eno2
addresses:
- 192.168.50.1/24

vlan60:
id: 60
link: eno2
addresses:
- 192.168.60.1/24

vlan70:
id: 70
link: eno2
addresses:
- 192.168.70.1/24

Potem aplikacja za pomocą komend:

sudo netplan try
sudo netplan apply
ip a
ip route

Przykładowe reguły firewalla:

```
sudo iptables -t nat -A POSTROUTING -o eno1 -j MASQUERADE
```

```
sudo iptables -A FORWARD -i vlan10 -o eno1 -j ACCEPT
```

```
sudo iptables -A FORWARD -i vlan20 -o eno1 -j ACCEPT
```

```
sudo iptables -A FORWARD -i vlan30 -o eno1 -j ACCEPT
```

```
sudo iptables -A FORWARD -i vlan40 -o eno1 -j ACCEPT
```

```
sudo iptables -A FORWARD -i vlan50 -o eno1 -j ACCEPT
```

```
sudo iptables -A FORWARD -i vlan60 -o eno1 -j ACCEPT
```

```
sudo iptables -A FORWARD -i vlan70 -o eno1 -j ACCEPT
```

```
sudo iptables -A FORWARD -i eno1 -o vlan10 -m state --state RELATED,ESTABLISHED -j ACCEPT
```

```
sudo iptables -A FORWARD -i eno1 -o vlan20 -m state --state RELATED,ESTABLISHED -j ACCEPT
```

```
sudo iptables -A FORWARD -i eno1 -o vlan30 -m state --state RELATED,ESTABLISHED -j ACCEPT
```

```
sudo iptables -A FORWARD -i eno1 -o vlan40 -m state --state RELATED,ESTABLISHED -j ACCEPT
```

```
sudo iptables -A FORWARD -i eno1 -o vlan50 -m state --state RELATED,ESTABLISHED -j ACCEPT
```

```
sudo iptables -A FORWARD -i eno1 -o vlan60 -m state --state RELATED,ESTABLISHED -j ACCEPT
```

```
sudo iptables -A FORWARD -i eno1 -o vlan70 -m state --state RELATED,ESTABLISHED -j ACCEPT
```

Potem aplikacja za pomocą komend:

```
sudo apt install iptables-persistent
```

```
sudo netfilter-persistent save
```

```
sudo systemctl enable netfilter-persistent
```

/etc/default/isc-dhcp-server

```
INTERFACESv4="vlan10 vlan20 vlan30 vlan40 vlan50 vlan60 vlan70"
```

```
INTERFACESv6=""
```

/etc/dhcp/dhcpd.conf

```
authoritative;
```

```
option domain-name "firma.local";
```

```
option domain-name-servers 192.168.1.11, 8.8.8.8;
```

```
default-lease-time 300;
```

```
max-lease-time 600;
```

```
one-lease-per-client true;
```

```
subnet 192.168.10.0 netmask 255.255.255.0 {
```

```
    range 192.168.10.100 192.168.10.200;
```

```
    option routers 192.168.10.3;
```

```
    option broadcast-address 192.168.10.255;
```

```
}
```

```
subnet 192.168.20.0 netmask 255.255.255.0 {
```

```
    range 192.168.20.100 192.168.20.200;
```

```
option routers 192.168.20.3;
option broadcast-address 192.168.20.255;
}
subnet 192.168.30.0 netmask 255.255.255.0 {
    range 192.168.30.100 192.168.30.200;
    option routers 192.168.30.3;
    option broadcast-address 192.168.30.255;
}
subnet 192.168.40.0 netmask 255.255.255.0 {
    range 192.168.40.100 192.168.40.200;
    option routers 192.168.40.3;
    option broadcast-address 192.168.40.255;
}
subnet 192.168.50.0 netmask 255.255.255.0 {
    range 192.168.50.100 192.168.50.200;
    option routers 192.168.50.3;
    option broadcast-address 192.168.50.255;
}
subnet 192.168.60.0 netmask 255.255.255.0 {
    range 192.168.60.100 192.168.60.200;
    option routers 192.168.60.3;
    option broadcast-address 192.168.60.255;
}
subnet 192.168.70.0 netmask 255.255.255.0 {
    range 192.168.70.100 192.168.70.200;
    option routers 192.168.70.3;
    option broadcast-address 192.168.70.255;
}
```

Potem aplikacja za pomocą komend:

```
sudo systemctl restart isc-dhcp-server
```

/etc/dnsmasq.conf

domain-needed
bogus-priv

interface=vlan10
interface=vlan20
interface=vlan30
interface=vlan40
interface=vlan50
interface=vlan60
interface=vlan70

listen-address=127.0.0.1
listen-address=192.168.10.1
listen-address=192.168.20.1
listen-address=192.168.30.1
listen-address=192.168.40.1
listen-address=192.168.50.1
listen-address=192.168.60.1
listen-address=192.168.70.1

server=/firma.local/192.168.1.11
server=8.8.8.8

Potem aplikacja za pomocą komend:

sudo systemctl restart dnsmasq

/etc/keepalived/keepalived.conf

```
vrrp_instance VI_1 {  
state MASTER  
interface eno2  
virtual_router_id 51  
priority 150  
advert_int 1
```

```
authentication {  
    auth_type PASS  
    auth_pass securepassword  
}
```

```
virtual_ipaddress {  
    192.168.1.3/24  
    192.168.10.3/24 dev vlan10  
    192.168.20.3/24 dev vlan20  
    192.168.30.3/24 dev vlan30  
    192.168.40.3/24 dev vlan40  
    192.168.50.3/24 dev vlan50  
    192.168.60.3/24 dev vlan60  
    192.168.70.3/24 dev vlan70
```



```
}  
}
```

Potem aplikacja za pomocą komend:

```
sudo systemctl restart keepalived  
ip a
```

Zaawansowane reguły firewall dla usług:

```
DC1="192.168.1.11"  
DC2="192.168.1.12"  
CLIENT_NETS="192.168.10.0/24 192.168.20.0/24 192.168.30.0/24 192.168.40.0/24 192.168.50.0/24  
192.168.60.0/24 192.168.70.0/24"
```

```
iptables -P FORWARD DROP  
iptables -A FORWARD -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT
```

```
for NET in $CLIENT_NETS; do  
for DC in $DC1 $DC2; do  
iptables -A FORWARD -s $NET -d $DC -p udp -m multiport --dports 53,88,123,389,464 -j ACCEPT  
iptables -A FORWARD -s $NET -d $DC -p tcp -m multiport --dports  
53,88,135,389,445,464,636,3268,3269 -j ACCEPT  
iptables -A FORWARD -s $NET -d $DC -p tcp --dport 49152:65535 -j ACCEPT  
done  
done  
  
SMB1="192.168.1.31"  
SMB2="192.168.1.32"
```

```
for NET in $CLIENT_NETS; do  
iptables -A FORWARD -s $NET -d $SMB1 -p tcp --dport 445 -j ACCEPT  
iptables -A FORWARD -s $NET -d $SMB2 -p tcp --dport 445 -j ACCEPT  
done
```

```
WAZUH1="192.168.1.21"  
WAZUH2="192.168.1.22"  
WAZUH3="192.168.1.24"
```

```
for NET in $CLIENT_NETS; do  
for WAZUH in $WAZUH1 $WAZUH2 $WAZUH3; do  
iptables -A FORWARD -s $NET -d $WAZUH -p tcp -m multiport --dports 443,514,1514,1515,9200 -j  
ACCEPT  
done  
done
```

```
MAIL="192.168.1.41"
```

```
for NET in $CLIENT_NETS; do
```

```
iptables -A FORWARD -s $NET -d $MAIL -p tcp -m multiport --dports
25,465,587,110,995,143,993,80,443 -j ACCEPT
```

done

```
for NET in $CLIENT_NETS; do
```

```
iptables -A FORWARD -s $NET -p udp --dport 53 -j ACCEPT
```

```
iptables -A FORWARD -s $NET -p tcp --dport 53 -j ACCEPT
```

```
iptables -A FORWARD -s $NET -p tcp -m multiport --dports 80,443,110,995,143,993,25,465,587 -j
ACCEPT
```

done

```
iptables -A INPUT -p vrrp -j ACCEPT
```

```
iptables -A INPUT -d 224.0.0.18 -j ACCEPT
```

```
iptables -A INPUT -i vlan10 -p udp --dport 67 -j ACCEPT
```

```
iptables -A INPUT -i vlan20 -p udp --dport 67 -j ACCEPT
```

```
iptables -A INPUT -i vlan30 -p udp --dport 67 -j ACCEPT
```

```
iptables -A INPUT -i vlan40 -p udp --dport 67 -j ACCEPT
```

```
iptables -A INPUT -i vlan50 -p udp --dport 67 -j ACCEPT
```

```
iptables -A INPUT -i vlan60 -p udp --dport 67 -j ACCEPT
```

```
iptables -A INPUT -i vlan70 -p udp --dport 67 -j ACCEPT
```

Komendy na blacklisty:

```
sudo apt install ipset
```

```
sudo ipset create blacklist hash:net family inet -exist
```

```
while read NET; do
```

```
    sudo ipset add blacklist "$NET" -exist
```

```
done < /etc/iptables/blacklist.txt
```

```
sudo iptables -I INPUT -m set --match-set blacklist src -j DROP
```

```
sudo iptables -I FORWARD -m set --match-set blacklist src -j DROP
```

Komendy na blokowanie po krajach:

```
sudo apt install xtables-addons-common
```

```
sudo iptables -I INPUT -m geoip --src-cc RU,CN -j DROP
```

```
sudo iptables -I FORWARD -m geoip --src-cc RU,CN -j DROP
```